

Novellm

An AI-Powered Reading Companion for Full-Book Q&A, Character Analysis, and Personalized Recommendations

Project Report

Tips Hindawi Challenge — June–July 2026

Full Name	Mariam Khaled Hassan
Project Name	Novellm
GitHub Username	mariamkh204
Challenge Batch	June–July 2026
Training Program	Large Language Models (LLMs) Program
Organization	Edrak for AI (Tips Hindawi)

1. Project Overview

Novellm is an AI-powered reading companion that turns any book PDF into an interactive, queryable experience. A reader uploads a book, and the system extracts and indexes the full text, then supports open-ended Q&A grounded in that text, spoiler-free "catch up" recaps, an automatically generated character map with an interactive relationship graph, excerpt summarization, comprehension quizzes, a plain-language explainer for dense passages, and a mood-based book recommender.

The project targets a real gap in existing reading tools: most reading assistants are either static summarizers or English-only. Novellm is built to work over full-length books rather than short passages, and to handle both digitally-native and scanned or Arabic-language PDFs through an automatic OCR fallback, extending usability to a wider range of readers and source material.

2. Problem Statement & Motivation

Readers picking up long or unfamiliar books face a few recurring frictions:

- Losing track of who characters are and how they relate to each other partway through a long book.
- Needing a spoiler-free recap after stepping away from a book for a while, without re-reading earlier chapters.
- Struggling with dense, archaic, or complex passages that require external explanation.
- Wanting book recommendations grounded in mood and reading preference rather than generic bestseller lists.
- Needing OCR support for scanned or Arabic-language texts, which most reading tools do not address.

Novellm addresses these by grounding every feature in the book's own text through retrieval-augmented generation (RAG), rather than relying on the model's general knowledge alone, and by packaging the features into a single dashboard rather than several disconnected tools.

3. System Architecture & Pipeline

The system is built as a single Kaggle GPU notebook with five stages:

- Model loading — Mistral-7B-Instruct-v0.2 is loaded from Hugging Face and quantized to 4-bit (NF4) with BitsAndBytes, keeping VRAM usage low enough to run on a single Kaggle GPU.
- Ingestion & indexing — PyMuPDF (fitz) extracts text page by page. If extraction yields little or no text (a scanned or image-based PDF), the pipeline automatically falls back to EasyOCR configured for Arabic and English.
- Chunking & retrieval — extracted text is split with LangChain's RecursiveCharacterTextSplitter (1000-character chunks, 150-character overlap, Arabic-aware separators) and embedded with a multilingual sentence-transformers model into a FAISS vector store for similarity search.
- Generation — a LangChain HuggingFacePipeline wrapper drives all LLM calls (Q&A, summaries, quizzes, recommendations, character extraction) through a single safe invocation helper that catches and reports generation errors instead of crashing the app.
- Interface — a custom-styled Gradio dashboard (dark theme, animated CSS, Plus Jakarta Sans typography) ties every feature to a button and tab, and the whole app is exposed publicly through pyngrok.

4. Feature Breakdown

4.1 Full-Book Retrieval-Augmented Q&A

Questions are answered by retrieving the top-k most similar chunks from the FAISS index and passing them as grounding context to the LLM, so answers are tied to the book's actual text rather than the model's prior knowledge.

4.2 Spoiler-Free Catch-Up Summaries

A slider lets the reader mark how far into the book they are. The system recaps only the chunks up to that point, generating a "Previously in this book..." summary that avoids revealing anything beyond the reader's current position.

4.3 Character Network & Relationship Graph

The full book is scanned to build a character registry: names, mention counts, roles, and short summaries. Roles (protagonist, supporting, minor) are assigned dynamically based on relative mention frequency rather than fixed rules. Relationships between characters are extracted from the LLM's output and normalized against the registry; where the model doesn't surface enough explicit relationships, a co-occurrence fallback scans paragraphs for characters mentioned together and infers an "Ally / Associate" link. The result is rendered as clickable character cards and an interactive force-directed graph (vis-network) embedded directly in the dashboard.

4.4 Excerpt Summarization & Quiz Generation

Any excerpt (or the whole book) can be condensed into a bulleted summary, or turned into a set of multiple-choice comprehension questions with collapsible answers for self-testing.

4.5 "Explain This" Plain-Language Mode

Dense or archaic passages can be pasted in and rephrased in clear, everyday language, including a breakdown of subtext or complex phrasing.

4.6 Personalized Book Recommender

Given a mood, pacing, ending type, protagonist archetype, length, and genre, the model suggests real books matching that profile, rendered as recommendation cards with a title, author, premise, and rationale for the fit.

4.7 Analytics Dashboard

A live sidebar reports total word count, estimated reading time, chunk count, and character count, alongside a bar chart of the most frequently mentioned characters, refreshed on demand.

5. Contribution Beyond the Core Labs

Beyond the RAG, summarization, and chaining techniques covered in the internship labs, Novellm adds a custom structured-output parser purpose-built to recover clean JSON from raw, sometimes malformed LLM text. It strips markdown code fences, attempts direct JSON parsing, falls back to regex-extracting the first

JSON array, and as a last resort applies `ast.literal_eval` — with a final normalization step that also accepts plain string outputs and coerces them into structured records. This parser is reused across three distinct features (character extraction, quiz generation, and book recommendations), turning otherwise unpredictable LLM text into UI-ready structured data. The character-relationship pipeline layered on top of it — registry resolution, duplicate-link detection, and a co-occurrence fallback for sparse relationship extraction — is itself a second novel component beyond anything covered in the labs.

6. Technologies Used

Component	Role
Mistral-7B-Instruct-v0.2	Core LLM, 4-bit NF4 quantized via BitsAndBytes for GPU efficiency
Transformers	Model loading, tokenizer, text-generation pipeline
LangChain	Orchestration, chunking (<code>RecursiveCharacterTextSplitter</code>), LLM pipeline wrapper
FAISS	Vector store for semantic search / retrieval (RAG)
Custom output parser	Reliable structured JSON extraction from raw LLM text
PyMuPDF, pypdf, EasyOCR	PDF text extraction, with Arabic/scanned-text OCR fallback
Gradio	Interactive dashboard UI (tabs, sidebar, custom CSS/animations)
pyngrok	Public tunneling for the Kaggle-hosted app

7. Results

- Built and ran a fully local, GPU-efficient RAG pipeline on a quantized 4-bit Mistral-7B model, keeping VRAM usage low enough for a single Kaggle GPU.
- Enabled full-book question answering grounded in the actual source text rather than the model's general knowledge.
- Added bilingual support (Arabic + English) with automatic OCR fallback, extending usability beyond digitally-native English PDFs.
- Delivered five distinct reading-assistant features — Q&A, catch-up summaries, character network analysis, quizzes/summarization, and recommendations — in a single unified dashboard.
- Automated character and relationship extraction from raw book text using a custom output parser, turning unstructured LLM output into structured, visualized data.
- Packaged the entire system into an interactive, shareable Gradio dashboard usable without local setup beyond a browser link.

8. Future Improvements

- Package the app as a mobile app (iOS/Android) for on-the-go reading companionship.

- Fine-tune the model on book-focused/literary datasets to improve domain knowledge and reduce generic responses.
- Add support for more file formats beyond PDF (EPUB, plain text, DOCX).
- Introduce user accounts with reading progress tracking synced across devices.
- Add audio/text-to-speech support so users can listen to summaries or explanations.

9. Conclusion

Novellm demonstrates a complete, self-contained RAG application built around a quantized open-weight LLM, combining retrieval-grounded Q&A, structured information extraction, and generative features into a single reading companion. Its bilingual OCR fallback and reusable structured-output parser extend it meaningfully beyond a standard summarization or Q&A demo, directly addressing the challenge's requirement for a genuine feature beyond the core lab material.